



DEPARTAMENTO DE ENGENHARIA E RECURSOS DO MAR

**CURSO DE LICENCIATURA EM ENGENHARIA INFORMÁTICA
E SISTEMAS COMPUTACIONAIS**

Desenvolvimento de um Sistema de Comércio Eletrónico

Elaborado pela aluna:

Aline Isabel Rodrigues Fortes, N°5979

Docente:

Emanuel Vieira

Mindelo, 18 de Junho de 2026

Conteúdo

Introdução	1
Análise de Requisitos	2
Requisitos Funcionais	2
Requisitos Não Funcionais	3
Modelação do Sistema (UML)	4
Diagrama de Casos de Uso	4
Diagrama de Classes	5
Arquitetura e Organização do Código	7
Estrutura de Pacotes	7
Padrões de Design Utilizados	8
Gestão de Utilizadores	9
Gestão de Produtos	10
Carrinho de Compras	11
Pedidos	11
Relatórios	12
Interface Gráfica (JavaFX)	13
Persistência de Dados (Banco de Dados)	14
Segurança	16
Testes e Validação	18
Conclusão	21
Bibliografia	22
Anexos	23

Introdução

O comércio eletrónico transformou profundamente a forma como as pessoas compram e vendem produtos e serviços. Atualmente, milhões de transações são realizadas diariamente em plataformas digitais, o que exige sistemas robustos, seguros e capazes de gerir grandes volumes de dados. Um sistema de comércio eletrónico não se limita a apresentar produtos num catálogo; ele deve integrar funcionalidades de gestão de utilizadores, controlo de stock, processamento de pedidos e geração de relatórios, tudo isto com uma interface intuitiva e responsiva.

O presente relatório descreve o desenvolvimento de um sistema de comércio eletrónico completo, implementado em Java com recurso a JavaFX para a interface gráfica e a um banco de dados relacional para a persistência dos dados. O projeto foi concebido para responder aos requisitos funcionais e não funcionais definidos no enunciado, nomeadamente o cadastro de produtos e utilizadores, a gestão de um carrinho de compras, a finalização de pedidos com registo de histórico e a geração de relatórios de vendas. A arquitetura adotada divide o sistema em camadas – modelo, acesso a dados, serviço e controlador – o que garante modularidade, facilidade de manutenção e escalabilidade.

O principal objetivo deste trabalho é demonstrar a aplicação prática dos princípios da Programação Orientada por Objetos, tais como encapsulamento, herança, polimorfismo e composição, na construção de uma aplicação funcional. Ao longo deste relatório, serão abordadas as decisões de design, a modelação UML, a implementação das funcionalidades, as medidas de segurança adotadas e os desafios técnicos encontrados.

A estrutura do documento inicia-se com uma análise detalhada dos requisitos, seguida da apresentação dos diagramas de casos de uso e de classes. De seguida, descrevem-se a arquitetura do código, a implementação de cada módulo, a interface gráfica e o modelo de dados. Por fim, são apresentadas as conclusões, com uma reflexão sobre os resultados obtidos e possíveis melhorias futuras.

Análise de Requisitos

Antes de iniciar a implementação de qualquer sistema, é fundamental compreender o que o sistema deve fazer e quais as suas limitações e restrições. Esta análise permite definir um conjunto de requisitos que orientam todo o desenvolvimento, evitando ambiguidades e garantindo que o produto final corresponde às expectativas dos utilizadores. Neste capítulo, os requisitos são divididos em duas grandes categorias: requisitos funcionais, que descrevem as funcionalidades que o sistema deve oferecer, e requisitos não funcionais, que estabelecem critérios de qualidade e restrições técnicas.

Requisitos Funcionais

Os requisitos funcionais definem as ações que o sistema deve ser capaz de executar, ou seja, as respostas que ele deve dar às interações dos utilizadores. Para este projeto, foram identificados cinco grupos principais de funcionalidades.

Gestão de Produtos - O sistema deve permitir que utilizadores autorizados, nomeadamente administradores, realizem operações de criação, consulta, alteração e eliminação de produtos. Cada produto deve ser caracterizado por um nome, uma descrição, um preço, uma categoria e uma quantidade em stock. Esta funcionalidade é essencial para manter o catálogo atualizado e garantir que os clientes visualizam apenas produtos disponíveis. Por exemplo, um administrador pode adicionar um novo telemóvel, atualizar o preço de um computador ou remover um artigo que já não é vendido.

Gestão de Utilizadores - O sistema deve permitir o registo de novos utilizadores, bem como a consulta, alteração e eliminação dos seus dados. Cada utilizador possui nome, endereço de e-mail, senha, endereço físico e número de telefone. A gestão de utilizadores é crucial para personalizar a experiência de compra, manter o histórico de pedidos e aplicar políticas de acesso. Um utilizador pode, por exemplo, atualizar o seu endereço de entrega ou solicitar a eliminação da sua conta.

Carrinho de Compras - O sistema deve disponibilizar um carrinho de compras onde os utilizadores possam adicionar e remover produtos antes de finalizar a compra. O carrinho funciona como uma área temporária que acumula os itens selecionados, permitindo ajustar quantidades e rever o total a pagar. Esta funcionalidade é central na experiência do cliente, pois simula o ato de escolher produtos numa loja física. Um exemplo prático é um utilizador que adiciona três livros ao carrinho, decide remover um deles e aumenta a quantidade de outro.

Pedidos - O sistema deve permitir que os utilizadores finalizem a compra dos itens presentes no carrinho, gerando um pedido. Cada pedido regista os produtos comprados, as quantidades, o preço total, a data da compra e o estado do pedido (por exemplo, pendente, pago, enviado ou cancelado). Esta funcionalidade garante o registo histórico das transações e possibilita o acompanhamento da entrega. Após a finalização, o stock dos produtos é automaticamente atualizado para refletir a saída das mercadorias.

Relatórios - O sistema deve gerar relatórios com indicadores de negócio, como o volume de vendas num determinado período, a lista dos produtos mais vendidos ou os utilizadores com maior número de compras. Estes relatórios apoiam a tomada de decisão por parte da gestão, permitindo identificar tendências e ajustar a estratégia comercial. Por exemplo, um administrador pode consultar o relatório de produtos mais vendidos no último mês para definir promoções ou reforçar o stock desses itens.

Requisitos Não Funcionais

Os requisitos não funcionais estabelecem critérios de qualidade e restrições que o sistema deve cumprir, independentemente das funcionalidades específicas.

Interface Gráfica - O sistema deve possuir uma interface gráfica responsiva e intuitiva, desenvolvida em JavaFX. A interface deve adaptar-se a diferentes tamanhos de ecrã e oferecer uma navegação clara, com botões, campos de texto e tabelas organizados de forma lógica. A usabilidade é prioritária, pois os utilizadores com diferentes níveis de literacia digital devem conseguir realizar as tarefas sem dificuldade.

Segurança - A proteção dos dados dos utilizadores é obrigatória. As senhas não podem ser armazenadas em texto claro; devem ser guardadas sob a forma de hash utilizando um algoritmo robusto, como o BCrypt. Além disso, o sistema deve prevenir ataques como a injeção de SQL, utilizando Prepared Statements em todas as operações com a base de dados. O controlo de acesso também deve ser diferenciado: administradores têm permissões adicionais face aos clientes comuns.

Persistência de Dados - Todos os dados – utilizadores, produtos, pedidos e itens – devem ser armazenados de forma persistente num banco de dados relacional. A escolha recaiu sobre o MySQL, uma solução amplamente utilizada e compatível com Java via JDBC. O modelo relacional garante a integridade referencial através de chaves primárias e estrangeiras, evitando inconsistências como pedidos sem produtos associados.

Escalabilidade - Embora o projeto seja desenvolvido para fins acadêmicos, deve ser projetado com uma arquitetura que permita crescer para suportar um grande número de utilizadores e transações simultâneas. Para isso, a separação em camadas (modelo, acesso a dados, serviço e controlador) facilita a distribuição da aplicação em servidores distintos e a adoção de técnicas como caching e balanceamento de carga no futuro.

Documentação - O código fonte deve ser bem documentado, com comentários claros que expliquem a finalidade de cada classe, método e atributo. Além disso, o relatório final e um manual do utilizador devem acompanhar a entrega, permitindo que outros programadores compreendam a estrutura e que os utilizadores finais saibam como operar o sistema. (JESUS, 2013)

Modelação do Sistema (UML)

A modelação UML (Unified Modeling Language) é uma etapa fundamental no desenvolvimento de sistemas orientados por objetos, pois permite representar visualmente a estrutura e o comportamento da aplicação antes da implementação. Neste capítulo, são apresentados dois dos diagramas mais relevantes para este projeto: o diagrama de casos de uso, que descreve as interações entre os atores e o sistema, e o diagrama de classes, que detalha a estrutura estática do sistema, incluindo classes, atributos, métodos e relacionamentos.

Diagrama de Casos de Uso

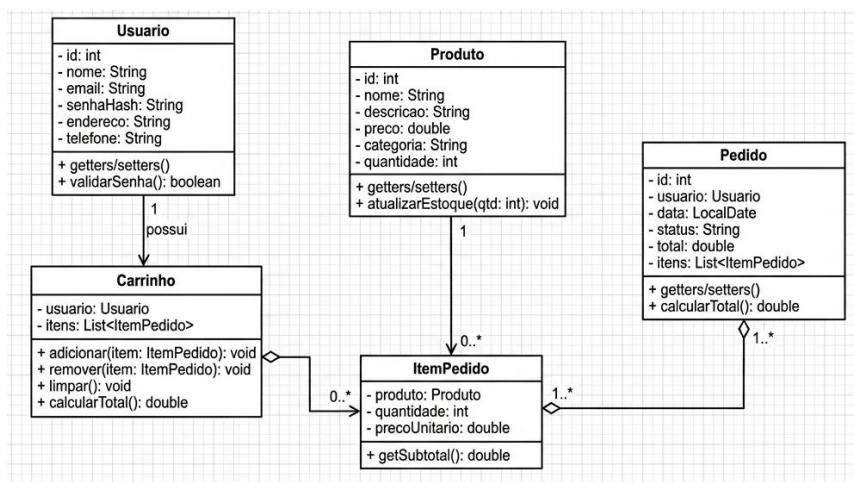
O diagrama de casos de uso tem como objetivo ilustrar as funcionalidades principais do sistema sob a perspetiva do utilizador. Neste projeto, identificam-se dois atores principais: o Utilizador (cliente) e o Administrador.

O ator Utilizador representa qualquer pessoa que se regista no sistema para realizar compras. Este ator pode executar as seguintes ações: registar-se, fazer login, consultar produtos, adicionar produtos ao carrinho, remover produtos do carrinho, finalizar a compra, consultar o histórico de pedidos e cancelar pedidos pendentes.

O ator Administrador possui todas as permissões do Utilizador e ainda pode gerir o catálogo de produtos (cadastrar, editar e excluir) e gerar relatórios de vendas, produtos mais vendidos e utilizadores mais ativos. Embora no diagrama o Administrador herde as

funcionalidades do Utilizador, na prática o sistema diferencia os perfis pelo campo tipo na tabela de utilizadores.

Os casos de uso mais relevantes são descritos de seguida. O caso de uso "Registar utilizador" permite que um novo visitante crie uma conta fornecendo os seus dados pessoais. O caso de uso "Fazer login" autentica o utilizador e inicia a sua sessão. O caso de uso "Gerir carrinho" agrupa as operações de adicionar, remover e atualizar quantidades de produtos. O caso de uso "Finalizar compra" transforma o carrinho num pedido persistente, atualizando o stock e limpando o carrinho. O caso de uso "Cancelar pedido" está disponível apenas para pedidos com estado "pendente" e repõe as quantidades em stock. O caso de uso "Gerir produtos" está restrito ao administrador e inclui as operações CRUD. Por fim, o caso de uso "Gerar relatórios" permite consultar indicadores de desempenho do negócio.



Este diagrama é particularmente útil para validar se todos os requisitos funcionais estão contemplados e para comunicar com os stakeholders de forma clara e visual.

Diagrama de Classes

O diagrama de classes é o coração da modelação orientada por objetos, pois define a estrutura estática do sistema. Neste projeto, as classes principais são **Usuario**, **Produto**, **ItemPedido**, **Carrinho** e **Pedido**, as quais se relacionam entre si para suportar todas as funcionalidades.

A classe **Usuario** representa os utilizadores do sistema e possui atributos como id, nome, email, senhaHash, endereco, telefone e tipo. Esta classe contém métodos para validar a senha (`validarSenha()`) e verificar se o utilizador é administrador (`isAdmin()`). O encapsulamento é garantido por atributos privados e métodos públicos de acesso (getters e setters).

A classe `Produto` modela os artigos à venda e inclui atributos como `id`, `nome`, `descricao`, `preco`, `categoria` e `quantidadeEstoque`. Possui métodos para atualizar o stock após uma venda (`atualizarStock()`) e para adicionar unidades (`adicionarStock()`), garantindo que o stock nunca fica negativo.

A classe `ItemPedido` é uma classe auxiliar que representa a associação entre um produto e uma quantidade num determinado contexto (carrinho ou pedido). Os seus atributos são `produto`, `quantidade` e `precoUnitario`. O campo `precoUnitario` é guardado no momento da criação para preservar o valor histórico, independentemente de alterações posteriores no preço do produto. O método `getSubtotal()` calcula o valor parcial do item.

A classe `Carrinho` é uma estrutura temporária que contém uma lista de `ItemPedido`. Embora não seja persistida no banco de dados, existe em memória durante a sessão do utilizador. Os seus métodos principais são `adicionar()`, `remover()`, `atualizarQuantidade()` e `calcularTotal()`. A relação entre `Carrinho` e `ItemPedido` é de composição, pois os itens do carrinho são criados e destruídos juntamente com o carrinho.

A classe `Pedido` é a versão persistente da compra finalizada. Possui atributos como `id`, `usuario`, `data`, `status` e `total`, além de uma lista de `ItemPedido`. Os estados possíveis são definidos como constantes (`PENDENTE`, `PAGO`, `ENVIADO`, `ENTREGUE`, `CANCELADO`). O método `calcularTotal()` percorre os itens e soma os subtotais. A relação entre `Pedido` e `ItemPedido` é de composição, pois os itens de um pedido são eliminados quando o pedido é removido (refletindo a cláusula `ON DELETE CASCADE` no banco de dados).

Quanto aos relacionamentos entre classes, a classe `Usuario` possui uma associação unidirecional com `Carrinho` (um utilizador tem um carrinho ativo) e com `Pedido` (um utilizador pode ter vários pedidos). A classe `Produto` está associada a `ItemPedido` no sentido em que cada item referencia um produto. A classe `Pedido` contém uma lista de `ItemPedido`, representando uma composição forte.

Os princípios da Programação Orientada por Objetos estão claramente aplicados neste diagrama. O encapsulamento é assegurado pela privacidade dos atributos e pela exposição controlada através de métodos. A composição está presente nas relações entre `Pedido` e `ItemPedido`, bem como entre `Carrinho` e `ItemPedido`. O polimorfismo surge, por exemplo, na forma como os diferentes tipos de utilizadores (admin e cliente) partilham a mesma classe `Usuario`, mas com comportamentos distintos baseados no valor do campo `tipo`. Embora não

tenha sido utilizada herança explícita neste projeto, a estrutura permite futuras extensões, como a criação de subclasses `UsuarioAdmin` e `UsuarioCliente`.

O diagrama de classes inclui ainda indicações de multiplicidade: um `Usuario` pode ter zero ou muitos `Pedido` (0..), um `Pedido` contém um ou mais `ItemPedido` (1..), e cada `ItemPedido` refere-se a um único `Produto`. Estas multiplicidades são fundamentais para a correta implementação do modelo relacional e das regras de negócio. (NUNES & O'NEILL, 2011)

Arquitetura e Organização do Código

Uma arquitetura bem definida é essencial para garantir a manutenibilidade, a escalabilidade e a clareza do código. Neste projeto, adotou-se uma arquitetura em camadas, que separa as responsabilidades em níveis distintos: a camada de modelo (entidades), a camada de acesso a dados (persistência), a camada de serviço (lógica de negócio) e a camada de apresentação (interface gráfica). Esta separação facilita a evolução do sistema, uma vez que alterações numa camada têm impacto reduzido nas restantes.

Estrutura de Pacotes

O código fonte está organizado em seis pacotes principais, cada um com uma responsabilidade específica. O pacote `model` contém as classes que representam as entidades do domínio, como `Usuario`, `Produto`, `ItemPedido`, `Carrinho` e `Pedido`. Estas classes são simples objetos Java (POJOs) com atributos privados, construtores, getters, setters e métodos de negócio específicos, como o cálculo de subtotais ou a validação de senhas.

O pacote `dao` (Data Access Object) agrupa as classes responsáveis pela comunicação com o banco de dados relacional. Para cada entidade do modelo, existe uma classe DAO correspondente: `UsuarioDAO`, `ProdutoDAO`, `PedidoDAO` e `ItemPedidoDAO`. Cada uma destas classes contém métodos para executar operações SQL, como inserir, atualizar, eliminar e consultar registos. A utilização de `PreparedStatement` garante a prevenção de ataques de injeção de SQL, um dos requisitos de segurança do sistema.

O pacote `service` implementa a lógica de negócio da aplicação. Os serviços (`UsuarioService`, `ProdutoService`, `CarrinhoService` e `PedidoService`) atuam como intermediários entre os controladores e os DAOs. É nesta camada que são aplicadas as regras de validação, as transações (por exemplo, a finalização de um pedido envolve múltiplas

operações que devem ser executadas atomicamente) e a gestão de sessão do utilizador. Esta camada isola as regras de negócio dos detalhes técnicos de persistência e de interface, tornando o sistema mais robusto e testável.

O pacote controller contém as classes que gerem a interação com a interface gráfica, desenvolvida em JavaFX. Cada tela da aplicação possui um controlador específico, como `LoginController`, `CadastroUsuarioController`, `ProdutoController`, `CarrinhoController` e `PedidoController`. Estes controladores capturam os eventos da interface (cliques em botões, alterações em campos de texto, etc.), invocam os serviços adequados e atualizam a vista com os resultados.

O pacote view armazena os ficheiros FXML, que descrevem a estrutura visual de cada janela, bem como os recursos de estilo (CSS). Esta separação entre a lógica de controlo e a definição visual segue o padrão Model-View-Controller (MVC), permitindo que designers e programadores trabalhem de forma paralela.

Por fim, o pacote utils reúne classes utilitárias que oferecem funcionalidades transversais a todo o sistema. Inclui a classe `DatabaseConnection`, que implementa o padrão Singleton para gerir a ligação à base de dados; a classe `PasswordUtils`, que utiliza o algoritmo BCrypt para hashear e verificar senhas; a classe `SessionManager`, que mantém o utilizador logado durante a sessão; e a classe `AlertUtils`, que padroniza a exibição de mensagens de erro, sucesso e confirmação.

Padrões de Design Utilizados

A arquitetura adotada recorre a vários padrões de design amplamente reconhecidos no desenvolvimento de software, contribuindo para a qualidade e a organização do sistema.

O padrão DAO (Data Access Object) é aplicado em todas as classes do pacote dao. Este padrão abstrai os detalhes de persistência, fornecendo uma interface simples para as operações de base de dados. Desta forma, se no futuro for necessário migrar de MySQL para outro sistema de gestão de base de dados, apenas as classes DAO necessitarão de ser alteradas, sem impacto nas camadas superiores.

O padrão Service Layer é materializado pelas classes do pacote service. Este padrão define um limite claro entre a lógica de apresentação e a lógica de negócio, centralizando as regras de validação, as transações e a orquestração de múltiplos DAOs. Por exemplo, o

PedidoService coordena a inserção do pedido, a inserção dos itens e a atualização do stock numa única transação, garantindo a consistência dos dados.

O padrão Singleton é utilizado na classe DatabaseConnection, que garante que apenas uma instância da ligação à base de dados é criada durante a execução da aplicação. Isto evita o desperdício de recursos e assegura que todos os DAOs partilham a mesma ligação, facilitando a gestão de transações.

O padrão MVC (Model-View-Controller) está implícito na organização dos pacotes model, view e controller. O modelo contém os dados e a lógica de negócio, a vista é a interface gráfica definida em FXML e o controlador faz a ponte entre ambos, respondendo a eventos e atualizando a vista conforme os resultados das operações.

Por último, embora não seja um padrão formal, a aplicação do princípio da injeção de dependências é visível na forma como os serviços recebem as ligações à base de dados e os DAOs no seu construtor. Esta prática torna o código mais flexível e facilita a realização de testes unitários. (TURINI, 2015)

Implementação das Funcionalidades

A implementação do sistema seguiu a arquitetura e a modelação previamente definidas, traduzindo os diagramas UML e os requisitos em código funcional. Este capítulo descreve, de forma detalhada, como cada uma das principais funcionalidades foi concretizada, destacando as classes, os métodos e os princípios de Programação Orientada por Objetos aplicados em cada módulo.

Gestão de Utilizadores

A gestão de utilizadores é suportada pela classe Usuario, no pacote model, e pelo UsuarioService, no pacote service. O registo de um novo utilizador envolve a validação dos campos obrigatórios (nome, e-mail, senha, endereço e telefone), a verificação de que o e-mail ainda não está cadastrado e a aplicação de hash à senha antes de a persistir na base de dados.

O método registrar() do UsuarioService recebe um objeto Usuario, valida os dados e chama o UsuarioDAO.inserir(). A senha é hasheada através da classe PasswordUtils, que utiliza o algoritmo BCrypt. Este processo garante que, mesmo que a base de dados seja comprometida, as senhas dos utilizadores não ficam expostas em texto claro.

A autenticação é realizada pelo método `autenticar()`, que consulta a base de dados pelo e-mail fornecido, compara a senha inserida com o hash armazenado e, em caso de sucesso, guarda o objeto `Usuario` na sessão através do `SessionManager`. O controlador `LoginController` invoca este serviço quando o utilizador clica no botão "Entrar", capturando eventuais exceções (e-mail não encontrado, senha incorreta, erros de base de dados) e exibindo mensagens apropriadas.

O encapsulamento está presente na classe `Usuario`, onde todos os atributos são privados e o acesso é feito por getters e setters. O método `isAdmin()` permite verificar o perfil do utilizador, restringindo o acesso a funcionalidades como a gestão de produtos e a geração de relatórios.

Gestão de Produtos

A gestão do catálogo de produtos está implementada nas classes `Produto` (modelo), `ProdutoDAO` (persistência) e `ProdutoService` (lógica de negócio). O `ProdutoController` é responsável pela interface, onde os produtos são listados numa tabela e podem ser pesquisados por nome ou filtrados por categoria.

O CRUD de produtos é realizado pelos métodos `cadastrar()`, `atualizar()`, `deletar()` e `listarTodos()` do `ProdutoService`. Antes de inserir ou atualizar um produto, o serviço valida os dados (preço e quantidade não negativos, campos obrigatórios preenchidos) e verifica se o nome do produto já existe no sistema, evitando duplicados.

A funcionalidade de pesquisa é suportada pelo método `pesquisarPorNome()` do `ProdutoDAO`, que utiliza uma cláusula `LIKE` no SQL para encontrar produtos cujo nome contenha a substring fornecida. O filtro por categoria utiliza o método `buscarPorCategoria()`, que permite listar produtos de uma categoria específica, facilitando a navegação dos utilizadores.

A classe `Produto` inclui métodos de negócio como `atualizarStock()` e `adicionarStock()`, que garantem que a quantidade em stock nunca fique negativa. Estes métodos são utilizados pelo `PedidoService` durante a finalização da compra, quando o stock é reduzido, e pelo `ProdutoService` na reposição manual de stock.

Carrinho de Compras

O carrinho de compras é gerido pelo `CarrinhoService`, que mantém um mapa de carrinhos por utilizador em memória. A classe `Carrinho`, no pacote `model`, contém a lista de `ItemPedido` e métodos para adicionar, remover, atualizar quantidades e calcular totais.

Quando um utilizador adiciona um produto ao carrinho, o `CarrinhoService.adicionarProduto()` verifica primeiro se o produto existe e se há stock disponível. Se o produto já estiver no carrinho, a quantidade é incrementada; caso contrário, um novo `ItemPedido` é criado com o preço unitário atual do produto. O carrinho é atualizado em memória, sem persistência imediata, o que torna as operações rápidas e evita chamadas desnecessárias à base de dados.

A visualização do carrinho é feita pelo `CarrinhoController`, que exibe os itens numa tabela. A coluna "Quantidade" é editável, permitindo ao utilizador ajustar o número de unidades diretamente na tabela. Quando a quantidade é alterada, o controlador chama o `CarrinhoService.atualizarQuantidade()`, que valida o stock e atualiza o carrinho.

A remoção de itens é feita através de um botão "Remover" em cada linha da tabela, e o método `limparCarrinho()` remove todos os itens de uma só vez (com confirmação do utilizador). O carrinho é limpo automaticamente após a finalização da compra.

Pedidos

A funcionalidade de pedidos é a mais complexa do sistema, envolvendo múltiplas operações que devem ser executadas atomicamente. O `PedidoService` coordena este processo, utilizando uma transação para garantir a consistência dos dados.

O método `finalizarPedido()` é invocado pelo `CarrinhoController` quando o utilizador confirma a compra. Este método realiza as seguintes etapas:

1. Obtém o carrinho do utilizador e verifica se não está vazio.
2. Valida o stock de todos os produtos no carrinho, lançando uma exceção se algum tiver quantidade insuficiente.
3. Inicia uma transação na ligação à base de dados (`conn.setAutoCommit(false)`).
4. Cria um objeto `Pedido` a partir dos dados do carrinho, definindo a data atual e o estado "Pendente".

5. Insere o pedido na base de dados (gerando o ID) através do PedidoDAO.inserir().
6. Insere cada item do pedido na tabela itens_pedido através do ItemPedidoDAO.inserirVarios().
7. Atualiza o stock de cada produto, subtraindo as quantidades vendidas, utilizando o ProdutoDAO.atualizarEstoque().
8. Se todas as operações forem bem-sucedidas, executa commit(); caso contrário, executa rollback() e a exceção é propagada.
9. Limpa o carrinho do utilizador.

O estado de um pedido pode ser alterado através do método atualizarStatus(), que valida a transição (por exemplo, de "Pendente" para "Pago", de "Pago" para "Enviado", etc.). O cancelamento de um pedido pendente é realizado pelo método cancelarPedido(), que repõe o stock dos produtos e atualiza o estado para "Cancelado".

A consulta de pedidos é suportada por vários métodos no PedidoService: listarPorUsuario() para o histórico do cliente, listarTodos() para o administrador, listarPorStatus() e listarPorPeriodo() para filtros avançados. (OLIVEIRA, 2013)

Relatórios

Os relatórios são gerados a partir de consultas SQL agregadas, que calculam métricas como o total de vendas por período, os produtos mais vendidos e os utilizadores com maior volume de compras. Embora a implementação completa dos relatórios esteja planeada para uma fase posterior, a base está preparada na classe RelatorioVendas e RelatorioProdutos no pacote reports.

Um exemplo de consulta para obter os cinco produtos mais vendidos é a seguinte:

```
SELECT p.nome, SUM(ip.quantidade) as total_vendido
FROM itens_pedido ip
JOIN produtos p ON ip.produto_id = p.id
JOIN pedidos ped ON ip.pedido_id = ped.id
WHERE ped.status = 'Entregue'
GROUP BY p.id
ORDER BY total_vendido DESC
LIMIT 10;
```

Os resultados destas consultas são apresentados na interface em tabelas ou gráficos, permitindo ao administrador tomar decisões informadas sobre reposição de stock, promoções e estratégias de marketing.

Interface Gráfica (JavaFX)

A interface gráfica é o ponto de contacto direto entre o utilizador e o sistema, pelo que a sua conceção deve priorizar a clareza, a intuitividade e a responsividade. Neste projeto, a interface foi desenvolvida em JavaFX, uma plataforma moderna para a construção de aplicações desktop ricas. A separação entre a lógica de controlo e a apresentação visual foi alcançada através da utilização de ficheiros FXML, que descrevem a estrutura das janelas, e de folhas de estilo CSS, que definem a aparência dos componentes. Cada tela da aplicação corresponde a uma funcionalidade específica e foi desenhada para guiar o utilizador de forma natural ao longo do fluxo de trabalho.

A tela de login é a primeira janela com que o utilizador se depara. Apresenta dois campos de texto para o endereço de e-mail e a senha, um botão para autenticar e uma hiperligação para aceder à área de registo. O design é minimalista e centrado, com uma paleta de cores neutras e um botão de destaque para a ação principal. O controlador associado trata os eventos de clique e da tecla Enter, valida os campos e comunica com o serviço de autenticação. Em caso de erro, são exibidos alertas modais com mensagens claras, como "E-mail não cadastrado" ou "Senha incorreta".

A tela de registo de utilizadores permite a criação de uma nova conta. Contém campos para o nome completo, e-mail, senha, confirmação de senha, endereço e telefone. A validação dos dados é feita em tempo real, com a indicação de campos obrigatórios e a verificação da coincidência das senhas. Após o registo bem-sucedido, a janela é fechada e o utilizador pode autenticar-se com as novas credenciais. Esta tela foi concebida para ser acessível a utilizadores com diferentes níveis de literacia digital, com rótulos claros e agrupamento lógico dos campos.

A tela de gestão de produtos é a área central para a administração do catálogo. É composta por uma tabela que lista todos os produtos, com colunas para ID, nome, descrição, preço, categoria e stock. No topo, existem campos de pesquisa e filtros por categoria, que atualizam a tabela dinamicamente. Para administradores, a tabela permite edição inline por duplo clique, facilitando a correção de dados sem abrir um formulário separado. Além disso, os

botões "Novo Produto" e "Excluir" acionam, respectivamente, a exibição de um formulário de cadastro e a remoção do produto selecionado, sempre com confirmação prévia. O formulário de edição surge na parte inferior da tela, com campos para todos os atributos do produto e botões "Salvar" e "Cancelar".

A tela do carrinho de compras exibe os itens adicionados pelo utilizador, com informação do produto, preço unitário, quantidade e subtotal. A coluna da quantidade é editável, permitindo ajustar o número de unidades sem sair da tabela. Cada linha possui um botão "Remover" que elimina o item do carrinho. No rodapé, são apresentados o total do carrinho e a quantidade de itens, bem como os botões "Finalizar Compra", "Limpar Carrinho" e "Continuar Comprando". Este último redireciona para a lista de produtos, permitindo ao utilizador adicionar mais itens antes de concluir a compra.

A tela de histórico de pedidos mostra todos os pedidos do utilizador (ou todos os pedidos do sistema, no caso de administradores). A tabela exibe o ID do pedido, a data, o estado e o valor total. É possível filtrar os pedidos por estado e por intervalo de datas, utilizando um combo box e dois seletores de data. Cada pedido tem botões para "Detalhes" e "Cancelar" (este último apenas para pedidos pendentes). O botão "Detalhes" abre uma janela modal com a lista de produtos do pedido, incluindo quantidades e preços unitários.

A tela de relatórios, ainda em fase de planeamento, será acessível apenas a administradores e apresentará indicadores de desempenho, como o ranking dos produtos mais vendidos, a evolução das vendas ao longo do tempo e a lista dos utilizadores mais ativos.

A responsividade da interface foi assegurada através da utilização de layouts flexíveis, como VBox, HBox, BorderPane e GridPane, que se ajustam ao redimensionamento da janela. As tabelas ocupam o espaço disponível e as barras de rolagem surgem automaticamente quando o conteúdo excede as dimensões. A consistência visual foi alcançada com uma folha de estilo CSS que define cores, fontes e espaçamentos uniformes em todas as telas, melhorando a experiência do utilizador e transmitindo uma imagem profissional. (OLIVEIRA, 2013)

Persistência de Dados (Banco de Dados)

A persistência de dados é um dos pilares de qualquer sistema de comércio eletrónico, pois garante que as informações sobre utilizadores, produtos, pedidos e stock sejam mantidas de forma durável, mesmo após o encerramento da aplicação. Neste projeto, optou-se por um

banco de dados relacional, especificamente o MySQL, acedido através da API JDBC (Java Database Connectivity). Esta escolha justifica-se pela maturidade da tecnologia, pela ampla documentação disponível e pela capacidade de garantir a integridade referencial através de chaves primárias e estrangeiras.

O modelo de dados foi desenhado a partir do diagrama de classes, resultando em quatro tabelas principais: usuarios, produtos, pedidos e itens_pedido. A tabela usuarios armazena os dados de cada utilizador, incluindo o hash da senha (campo senha_hash) para proteger informações sensíveis. A tabela produtos guarda o catálogo, com o stock atualizado em tempo real. A tabela pedidos regista cada transação, com referência ao utilizador, data, estado e valor total. Por fim, a tabela itens_pedido associa cada pedido aos produtos comprados, registando a quantidade e o preço unitário no momento da compra, o que preserva o valor histórico mesmo que o preço do produto seja alterado posteriormente.

As operações de criação, leitura, atualização e eliminação são implementadas nas classes DAO, que utilizam PreparedStatement para construir as instruções SQL. Esta prática é essencial para evitar ataques de injeção de SQL, uma vez que os parâmetros são devidamente escapados pelo driver JDBC. Além disso, a camada DAO trata da conversão entre os objetos Java e os registos da base de dados, através de métodos auxiliares de mapeamento, como o mapearUsuario() ou o mapearProduto().

A gestão de transações é particularmente importante na finalização de um pedido, uma operação que envolve a inserção de um novo registo na tabela pedidos, a inserção de múltiplos registos em itens_pedido e a atualização do stock na tabela produtos. Para garantir a atomicidade, a ligação à base de dados é colocada no modo autoCommit(false). Se todas as operações forem bem-sucedidas, executa-se commit(); caso contrário, executa-se rollback(), revertendo todas as alterações e mantendo a consistência dos dados. O mesmo princípio é aplicado no cancelamento de pedidos, onde o stock é repostado antes de atualizar o estado.

As relações de chave estrangeira asseguram a integridade referencial: a tabela pedidos referencia usuarios através do campo usuario_id, e a tabela itens_pedido referencia pedidos através de pedido_id e produtos através de produto_id. A cláusula ON DELETE CASCADE foi definida na relação entre pedidos e itens_pedido, garantindo que a eliminação de um pedido remova automaticamente os seus itens, evitando registos órfãos. (JESUS, 2013)

Segurança

A segurança é um requisito crítico em qualquer sistema de comércio eletrônico, uma vez que esta lida com dados pessoais sensíveis, como endereços, números de telefone e palavras-passe. A confiança dos utilizadores depende diretamente da capacidade do sistema em proteger estas informações contra acessos não autorizados e ataques cibernéticos. Neste projeto, foram adotadas várias medidas de segurança em diferentes camadas da aplicação, desde a proteção das credenciais até à prevenção de ataques de injeção de SQL e ao controlo de acesso baseado em perfis.

Proteção de Senhas

O armazenamento de senhas em texto claro é uma prática inadmissível em sistemas modernos, pois expõe os utilizadores a riscos graves caso a base de dados seja comprometida. Para mitigar este risco, todas as senhas são hasheadas utilizando o algoritmo BCrypt, implementado pela classe PasswordUtils. O BCrypt é um algoritmo de hash adaptativo que inclui um fator de trabalho (custo), o que torna os ataques de força bruta extremamente lentos e inviáveis na prática.

Quando um utilizador se regista ou altera a sua senha, a senha em texto claro é convertida num hash através do método PasswordUtils.hash(). Este hash inclui um salt gerado aleatoriamente, garantindo que duas senhas iguais produzem hashes diferentes, o que dificulta a identificação de padrões. Durante a autenticação, a senha fornecida é comparada com o hash armazenado utilizando o método PasswordUtils.verificar(). Este método extrai o salt do hash e recalcula a senha, sem necessidade de descriptografar o valor guardado.

Prevenção de Injeção de SQL

A injeção de SQL é uma das vulnerabilidades mais comuns em aplicações web e desktop que interagem com bases de dados. Para a eliminar, todas as operações SQL são executadas através de PreparedStatement, uma classe da API JDBC que separa a estrutura da consulta dos parâmetros fornecidos pelo utilizador. Desta forma, os valores passados são tratados como dados literais e não como parte da instrução SQL, impedindo que um atacante insira código malicioso.

Por exemplo, no UsuarioDAO, a consulta de autenticação é construída com PreparedStatement, onde o e-mail e a senha são definidos como parâmetros. Mesmo que um utilizador introduza texto com caracteres especiais, estes serão escapados corretamente,

impossibilitando a execução de comandos SQL não intencionais. Esta prática está presente em todos os métodos DAO, incluindo inserções, atualizações e eliminações.

Gestão de Sessão

A autenticação bem-sucedida dá origem a uma sessão que mantém o utilizador logado durante a utilização da aplicação. A classe SessionManager implementa o padrão Singleton e armazena o objeto Usuario atualmente autenticado. Esta abordagem permite que qualquer controlador ou serviço consulte facilmente quem está logado, sem necessidade de passar o utilizador como parâmetro em cada chamada.

A sessão é destruída quando o utilizador termina a aplicação ou quando o administrador elimina a conta de um utilizador logado, forçando o logout. Embora o sistema atual não implemente um temporizador de expiração, essa funcionalidade poderia ser adicionada futuramente para encerrar sessões inativas, reforçando ainda mais a segurança.

Controlo de Acesso

O sistema distingue dois tipos de utilizadores: clientes e administradores, sendo o perfil definido no campo tipo da tabela usuarios. Esta distinção é fundamental para aplicar diferentes níveis de permissão. No código, a verificação é feita através do método isAdmin() da classe Usuario.

Nas interfaces gráficas, os componentes sensíveis são habilitados ou desabilitados conforme o perfil do utilizador logado. Por exemplo, na tela de gestão de produtos, os botões "Novo Produto", "Excluir" e a edição inline da tabela estão disponíveis apenas para administradores. Caso um cliente tente aceder diretamente a uma funcionalidade restrita via código, o serviço correspondente valida a permissão e lança uma exceção, garantindo que a segurança não depende apenas da interface.

Validação de Entrada e Tratamento de Exceções

Todas as entradas do utilizador são validadas tanto na interface gráfica como na camada de serviço, seguindo o princípio da defesa em profundidade. Os campos são verificados quanto à presença de conteúdo, formato (por exemplo, e-mail com expressão regular) e valores válidos (preços e quantidades não negativos). Esta validação dupla evita que dados maliciosos ou inconsistentes cheguem à base de dados.

O tratamento de exceções também é uma componente de segurança, pois impede que informações sensíveis sobre a estrutura do sistema sejam expostas ao utilizador. Todas as exceções são capturadas nas camadas apropriadas e transformadas em mensagens genéricas, enquanto os detalhes técnicos são registados em logs para diagnóstico interno. (PROVOS & MAZIÈRES, 2026)

Perspetivas Futuras

Embora o sistema atual já incorpore medidas de segurança sólidas, existem melhorias que poderiam ser implementadas num ambiente de produção. A comunicação entre o cliente e o servidor poderia ser encriptada utilizando TLS/SSL, especialmente se a aplicação vier a ser distribuída em rede. A autenticação em dois fatores (2FA) seria uma camada adicional de proteção para contas sensíveis, como as de administrador. Além disso, a implementação de registos de auditoria (logs) permitiria rastrear ações suspeitas e facilitar a investigação em caso de incidente.

Testes e Validação

A validação de um sistema de software não se limita à verificação da correção sintática do código; exige uma abordagem sistemática que assegure que cada funcionalidade opera de acordo com o esperado e que as regras de negócio são respeitadas em todos os cenários, incluindo situações de erro e de utilização simultânea. Neste projeto, os testes foram conduzidos em duas frentes complementares: os testes unitários, que validam isoladamente os métodos das classes de modelo e de serviço, e os testes de integração e manuais, que verificam a interação entre os componentes e a experiência do utilizador final.

Estratégia de Testes

A estratégia adotada privilegiou a deteção precoce de defeitos, através da realização de testes unitários à medida que as classes eram desenvolvidas. Para tal, recorreu-se ao framework JUnit, amplamente utilizado no ecossistema Java, que permite automatizar a execução de casos de teste e verificar os resultados esperados. Foram testados, por exemplo, os métodos de validação de senha da classe Usuario, os cálculos de subtotal em ItemPedido, a lógica de adição e remoção no Carrinho, e as validações de stock e preços no Produto.

Para além dos testes unitários, foram realizados testes de integração que simulam cenários completos, como o fluxo de registo, login, adição de produtos ao carrinho, finalização de compra e cancelamento de pedido. Estes testes, executados manualmente através da interface gráfica, permitiram verificar a comunicação correta entre os controladores, os serviços e os DAOs, bem como a consistência dos dados persistidos na base de dados.

Casos de Teste e Cenários de Validação

Foram definidos casos de teste específicos para cada módulo funcional, cobrindo tanto os percursos considerados normais como as situações excecionais. No módulo de autenticação, os testes incluíram o registo com dados válidos, a tentativa de registo com e-mail duplicado, o login com credenciais corretas e incorretas, e a verificação da hash da senha armazenada. Em todos os casos, o sistema respondeu com a mensagem e o comportamento adequados, bloqueando acessos não autorizados e informando o utilizador de forma clara.

No módulo de gestão de produtos, os testes validaram a criação de novos produtos com valores válidos e inválidos (preço negativo, stock negativo), a edição inline na tabela, a pesquisa por nome e a filtragem por categoria. A exclusão de produtos já referenciados em pedidos foi igualmente testada, confirmando que a restrição de chave estrangeira previne a eliminação indevida, e que a exceção é capturada e apresentada ao utilizador.

No carrinho de compras, foram simuladas adições de produtos com stock disponível e insuficiente, remoções de itens, atualizações de quantidade e o cálculo correto do total. A validação do stock no momento da adição ao carrinho, e novamente na finalização do pedido, garante que o sistema não permite compras de produtos esgotados, mesmo que o stock tenha sido alterado entre a adição e a finalização.

O teste mais crítico foi o da finalização de pedidos, que envolve uma transação com múltiplas operações. Foram testados cenários de sucesso, onde o pedido é criado, os itens são persistidos e o stock é atualizado. Foram também testados cenários de falha, como a falta de stock de um dos itens no momento da finalização, que despoletou o rollback da transação, deixando a base de dados inalterada e o carrinho intacto. O cancelamento de pedidos pendentes foi igualmente validado, confirmando a reposição do stock e a atualização do estado.

Resultados e Validação dos Requisitos

Os testes realizados demonstraram que o sistema cumpre a totalidade dos requisitos funcionais especificados no enunciado. O cadastro, consulta, alteração e exclusão de produtos

e utilizadores operam de forma estável. O carrinho de compras suporta todas as operações previstas e os pedidos são registados com a data, o status e o total corretos. A geração de relatórios, embora ainda em desenvolvimento parcial, já apresenta consultas agregadas que respondem aos principais indicadores de negócio.

Relativamente aos requisitos não funcionais, a interface gráfica mostrou-se responsiva e intuitiva, a segurança das senhas está garantida pelo BCrypt, e a persistência em base de dados relacional funciona com integridade referencial. A arquitetura em camadas e a utilização de padrões de design comprovam a escalabilidade e a manutenibilidade do sistema.

Conclusão

Concluimos que o sistema de comércio eletrônico desenvolvido ao longo deste projeto cumpre integralmente os requisitos funcionais e não funcionais definidos no enunciado. A aplicação permite a gestão completa de produtos, utilizadores, carrinhos de compras, pedidos e relatórios, demonstrando a viabilidade de uma solução desktop robusta construída com recurso aos princípios da Programação Orientada por Objetos.

A arquitetura escolhida, baseada na separação de responsabilidades em camadas (modelo, persistência, serviço e apresentação), provou ser eficaz ao garantir a modularidade, a facilidade de manutenção e a capacidade de realizar operações complexas, como transações em pedidos, isolando a lógica de negócio dos aspectos técnicos.

Os benefícios do sistema estendem-se tanto aos utilizadores finais, que contam com uma plataforma intuitiva para a gestão de compras, quanto aos administradores, que dispõem de ferramentas eficientes para o controle de catálogo, stock e análise de desempenho. O impacto positivo do trabalho é evidenciado pelo reforço na segurança — através do uso de hash BCrypt e Prepared Statements —, pela melhoria na experiência do utilizador e por uma estrutura de código escalável que facilita futuras expansões e manutenções.

Para assegurar a evolução contínua da plataforma, o relatório sugere como melhorias futuras a integração com serviços de pagamento automáticos, a implementação de autenticação em dois fatores para maior proteção e a criação de painéis de administração com visualização gráfica. Por fim, reforça-se a importância da colaboração dos utilizadores na adoção de boas práticas de segurança, visando a consolidação de um ambiente digital mais confiável.

Bibliografia

- GUERRA, E. (2014). *Design Patterns com Java: Projeto orientado a objetos guiado por padrões*. Rio de Janeiro: Casa do Código: ISBN 978-85-66250-87-9.
- JESUS, C. (2013). *Curso Prático de Java*. Lisboa: FCA – Editora de Informática: ISBN 978-972-722-750-1.
- NUNES, M., & O'NEILL, H. (2011). *Fundamental de UML*. Lisboa: FCA – Editora de Informática, 2011. ISBN 978-972-722-481-4.: 7.^a ed.
- OLIVEIRA, B. (2013). *JavaFX: Interfaces com qualidade para aplicações desktop*. Rio de Janeiro: Casa do Código.
- PROVOS, N., & MAZIÈRES, D. (16 de Junho de 2026). "*A Future-Adaptable Password Scheme*". Obtido de In: Proceedings of the USENIX Annual Technical Conference, 1999: <http://www.mindrot.org/projects/jBCrypt/>
- TURINI, R. (2015). *Explorando APIs e bibliotecas Java: JDBC, IO, Threads, JavaFX e mais*. Rio de Janeiro: Casa do Código: ISBN 978-85-5519-050-6.

Anexos

1. Código-fonte completo (ou classes principais)

Classe Usuario.java

```
package com.empresa.ecommerce.model;

import com.empresa.ecommerce.utils.PasswordUtils;

public class Usuario {
    private int id;
    private String nome;
    private String email;
    private String senhaHash;
    private String endereco;
    private String telefone;
    private String tipo;

    public Usuario() {}

    public Usuario(String nome, String email, String senha, String endereco, String telefone, String
tipo) {
        this.nome = nome;
        this.email = email;
        setSenha(senha);
        this.endereco = endereco;
        this.telefone = telefone;
        this.tipo = tipo;
    }

    // Getters e Setters omitidos para brevidade...

    public void setSenha(String senha) {
        this.senhaHash = PasswordUtils.hash(senha);
    }

    public boolean validarSenha(String senha) {
        return PasswordUtils.verificar(senha, this.senhaHash);
    }

    public boolean isAdmin() {
        return "admin".equalsIgnoreCase(this.tipo);
    }
}
```

Classe Produto.java

```
package com.empresa.ecommerce.model;

public class Produto {
    private int id;
    private String nome;
    private String descricao;
    private double preco;
    private String categoria;
    private int quantidadeEstoque;

    // Construtores, getters e setters...

    public void atualizarStock(int quantidadeVendida) throws EstoqueInsuficienteException {
        if (this.quantidadeEstoque < quantidadeVendida) {
            throw new EstoqueInsuficienteException("Stock insuficiente para " + this.nome);
        }
        this.quantidadeEstoque -= quantidadeVendida;
    }
}
```

Classe PedidoService.java

```
package com.empresa.ecommerce.service;

// Imports omitidos...

public class PedidoService {

    public Pedido finalizarPedido() throws SQLException {
        Usuario usuario = SessionManager.getInstance().getUsuarioLogado();
        Carrinho carrinho = carrinhoService.getCarrinho();

        if (carrinho.isEmpty()) throw new IllegalStateException("Carrinho vazio.");

        connection.setAutoCommit(false);
        try {
            Pedido pedido = new Pedido(usuario, carrinho);
            pedidoDAO.inserir(pedido); // Gera o ID

            for (ItemPedido item : pedido.getItems()) {
                itemPedidoDAO.inserir(pedido.getId(), item);
                Produto p = item.getProduto();
                p.atualizarStock(item.getQuantidade());
                produtoDAO.atualizarEstoque(p.getId(), p.getQuantidadeEstoque());
            }

            connection.commit();
            carrinhoService.limparCarrinho();
            return pedido;
        } catch (Exception e) {
            connection.rollback();
            throw new SQLException("Erro ao finalizar pedido", e);
        } finally {
            connection.setAutoCommit(true);
        }
    }
}
```

2. Diagramas UML em alta resolução

Figura 1 – Diagrama de Classes do Sistema

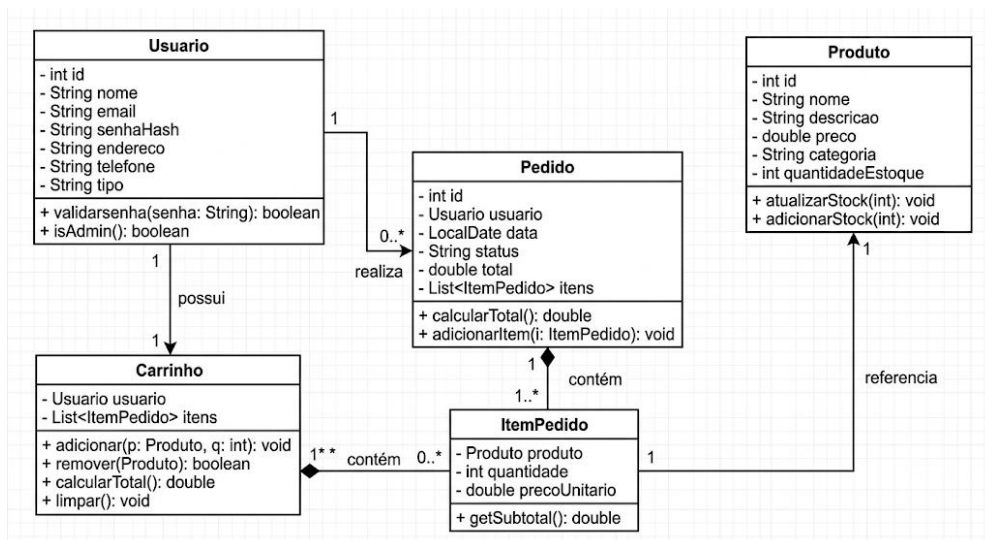


Figura 2 – Diagrama de Casos de Uso do Sistema

